VISVESVARAYA TECHNOLOGICAL UNIVERSITY
BELAGAVI-590018

Computer Vision Project Report

on

MORPHOLOGICAL IMAGE PROCESSING OPERATIONS

Submitted as part of the assignment requirements for the 6th semester of the
Bachelor of Engineering in Computer Science and Engineering and
affiliated with Visvesvaraya Technological University, Belagavi

Submitted by

Raghu Charan R A	**1RN23CS412**
Rajkumar	**1RN23CS413**
T Sharath	**1RN23CS416**

Under the Guidance of

Mrs. Savitha T
Assistant Professor
Dept. of CSE, RNSIT

Department of Computer Science and Engineering
RNS Institute of Technology
Affiliated to VTU, Recognized by GOK, Approved by AICTE,New Delhi
NACC 'A + Grade' Accredited, NBA Accredited (UG-CSE, ECE, ISE, EIE and EEE)
Channasandra, Dr.Vishnuvardhan Road, Bengaluru-560098
Ph:(080)28611880, 28611881 URL:www.rnsit.ac.in

2024-2025

RN SHETTY TRUST®

RNS INSTITUTE OF TECHNOLOGY

Affiliated to VTU, Recognized by GOK, Approved by AICTE, New Delhi
NACC 'A + Grade' Accredited, NBA Accredited (UG-CSE,ECE,ISE,EIE and EEE)
Channasandra, Dr.Vishnuvardhan Road, Bengaluru-560098
Ph:(080)28611880,28611881 URL:www.rnsit.ac.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

Certified that the Project work entitled **MORPHOLOGICAL IMAGE PROCESSING OPERATIONS** has been successfully carried out at **RNSIT** by **Raghu Charan R A** bearing **1RN23CS412** ,and **Rajkumar** bearing **1RN23CS413** ,and **T Sharath** bearing **1RN23CS416** bonafide students of **RNS Institute of Technology**. Submitted as part of the assignment requirements for the 6th semester of the Bachelor of Engineering in Computer Science and Engineering and affiliated with Visvesvaraya Technological University, Belagavi **2024-2025**. The Project report has been approved as it satisfies the academic requirements in respect of project work for the said degree.

_____

Mrs. Savitha T
Asst. Professor
Dept. of CSE, RNSIT

_____

Dr. Kavitha C
Dean and Head
Dept. of CSE , RNSIT

Abstract

This project introduces a web-based application that enables users to perform morphological image processing operations on uploaded images, offering an interactive and educational experience in the field of computer vision. Developed using the Flask web framework and OpenCV library, the application allows users to apply fundamental morphological transformations such as Erosion, Dilation, Opening, Closing, Hit, and Miss. These operations are vital for analyzing and manipulating the structure of objects in binary and grayscale images, with applications ranging from noise reduction and shape refinement to pattern detection. The system lets users control the kernel size, which influences the extent and nature of each transformation, thereby offering flexibility and deeper understanding of image structure manipulation. For instance, Erosion removes small white noises by shrinking object boundaries, while Dilation emphasizes features by expanding these boundaries. Opening and Closing operations help smooth object contours and eliminate minor defects, whereas Hit-or-Miss transformation is used to detect specific shape patterns. The application provides a side-by-side comparison of the original and processed images, enhancing user comprehension through immediate visual feedback. Its intuitive interface makes it accessible to students, researchers, and professionals, eliminating the need for complex programming skills or software installation. By bringing powerful morphological tools to the browser, this project bridges theoretical knowledge and practical implementation, making it ideal for educational demonstrations and initial experimentation in shape-based image analysis. Future developments may include more advanced operations, multi-channel image support, and domain-specific presets for tasks such as medical imaging or industrial quality control. Ultimately, this project highlights the importance and versatility of morphological techniques in digital image processing and presents a practical platform for exploring their real-world potential.

Acknowledgement

I extend my profound thanks to the **Management of RNS Institute of Technology** for fostering an environment that promotes innovation and academic excellence.

I want to express my gratitude to our beloved Director, **Dr. M K Venkatesha**, and Principal, **Dr. Ramesh Babu H S** , for their constant encouragement and insightful support.Their guidance has been pivotal in keeping me motivated throughout this endeavour.

My hearfelt appreciation goes to **Dr. Kavitha C**, Dean and HoD of Computer Science and Engineering, for her vital advice and constructive feedback, which significantly contributed to shaping this project.

I am deeply grateful to project guide, **Mrs. Savitha T**, Assistant Professor, for their unwavering support, guidance, and valuable suggestions throughout the complition of this project.

Raghu Charan R A (1RN23CS412)

Rajkumar (1RN23CS413)

T Sharath (1RN23CS416)

Contents

Abstract **i**

Acknowledgement **ii**

List of Figures **v**

1 INTRODUCTION **1**

1.1 Problem Statement 1

1.2 Objectives 2

1.3 Scope 2

2 Metholodogy **3**

2.1 Software Requirements 3

2.2 Algorithms 4

2.3 Pseudocode 5

2.4 Prediction Algorithm 6

2.5 Data Collection and Analysis Procedure 7

2.5.1 Data Collection Procedure 7

2.5.2 Data Analysis Procedure 9

3 IMPLEMENTATION **11**

3.1 Frontend code (index.html) 11

3.2 Backend Code (app.py) 13

4 RESULT **16**

4.1 Input 16

4.2 Output 17

4.3 Outcome 20

5 CONCLUSION **21**

5.1 Conclusion 21

5.2 Future Enhancements 22

References **23**

List of Figures

4.1 Input car.png 16
4.2 Output of Erosion 17
4.3 Output of Dilation 17
4.4 Output of Opening 18
4.5 Output of Closing 18
4.6 Output of Hit 19
4.7 Output of Miss 19

Chapter 1

INTRODUCTION

Morphological image processing is a key technique in the field of computer vision, used to analyze and process images based on their shapes and structures. It is particularly effective for tasks such as noise removal, object separation, and feature extraction in binary and grayscale images. This project focuses on developing a web-based application that allows users to apply various morphological operations—such as Erosion, Dilation, Opening, Closing, Hit, and Miss—directly to uploaded images. Built using Flask and OpenCV, the application provides a user-friendly interface where users can adjust parameters like kernel size and instantly view the results. By processing and displaying the original and modified images side by side, the tool offers a clear visual understanding of how each operation affects image structure. The primary goal is to create an educational and accessible platform for experimenting with morphological transformations, useful for students, researchers, and anyone interested in digital image processing.

1.1 Problem Statement

Understanding and applying morphological image processing techniques can be difficult without practical, visual tools. Most existing solutions require programming skills or complex software setups. This creates a barrier for students, educators, and beginners in computer vision. There is a need for a simple, web-based tool to demonstrate and explore morphological operations interactively.

1.2 Objectives

- To develop a user-friendly web application for performing morphological image processing operations.
- To enable users to upload images and apply transformations like Erosion, Dilation, Opening, Closing, Hit, and Miss.
- To provide real-time visual feedback by displaying original and processed images side by side.
- To make morphological concepts accessible to students, educators, and beginners without requiring programming knowledge.
- To support adjustable parameters (e.g., kernel size) for flexible experimentation and deeper understanding.

1.3 Scope

- Allows users to upload and process images using basic morphological operations.
- Provides an interactive web interface built with Flask and OpenCV.
- Supports common operations like Erosion, Dilation, Opening, Closing, Hit, and Miss.
- Enables adjustment of kernel size to observe different transformation effects.
- Offers real-time visualization of both original and processed images for comparison.
- Designed for educational and demonstration purposes, targeting students and beginners in image processing.

  The scope does not belongs

- Does not include advanced morphological techniques like skeletonization or watershed segmentation.
- No support for video processing or real-time camera feed input.
- No user authentication or multi-user management features.

Chapter 2

Methodology

2.1 Software Requirements

1. Programming Language

- Python 3.x
  - Python is chosen for its simplicity and robust ecosystem for web and image processing development.
  - Compatible with libraries like OpenCV and Flask, making it ideal for this project.

2. Development Environment

- PyCharm / Visual Studio Code / Jupyter Notebook
  - These IDEs support efficient coding, debugging, and project management.
  - Jupyter is useful for testing image transformations in standalone mode.

3. Libraries and Frameworks

- Flask
  - A lightweight Python web framework for creating web servers and routing.
  - Enables dynamic content rendering and session management.
- OpenCV (cv2)

  - Core library for performing morphological operations like Erosion, Dilation, Opening, Closing, Hit, and Miss.
  - Supports image reading, writing, thresholding, and structural transformation.

- NumPy
  - Handles image data as multidimensional arrays.
  - Essential for kernel creation and pixel-level manipulation.

- Werkzeug Jinja2 (built into Flask)
  - Used for secure file handling and rendering HTML templates dynamically.

4. Package Manager

- pip
  - Used to install required libraries such as Flask, OpenCV, and NumPy via the command:pip install flask opencv-python numpy

5. Operating System

- Windows 10/11 or Linux (Ubuntu/Debian)
  - Compatible with all project dependencies.
  - Linux is often preferred for better performance in web hosting and image processing tasks.

2.2 Algorithms

Image Upload and Retrieval

- Purpose : Retrieve the uploaded image file from the client and save it to the server.

Steps:

1. Check if an image file is received in the POST request.
2. Secure the filename and store it in a predefined folder.

3. Store the image path in a session for further access.

Morphological Operation Selection and Execution

- Purpose : Retrieve user inputs (operation and kernel size), apply the selected morphological operation, and save the output image.

Steps:

1. Read the selected operation and kernel size from the form.
2. Load the image using OpenCV.
3. Based on the selected operation, apply the corresponding OpenCV function.
4. Save the processed image to the server for display.

2.3 Pseudocode

```
# Image Upload and Form Handling
IF request method is POST:
    image_file = get uploaded image
    IF image_file exists:
        filename = secure_filename(image_file)
        save image to 'static/uploads' folder
        store image path in session

# Retrieve User Input
operation = get selected operation from form
kernel_size = get selected kernel size from form

# Read and Process Image
image = cv2.imread(session_image_path)
kernel = np.ones((kernel_size, kernel_size), np.uint8)
```

```
IF operation == 'Erosion':
    result = cv2.erode(image, kernel)
ELSE IF operation == 'Dilation':
    result = cv2.dilate(image, kernel)
ELSE IF operation == 'Opening':
    result = cv2.morphologyEx(image, MORPH_OPEN, kernel)
ELSE IF operation == 'Closing':
    result = cv2.morphologyEx(image, MORPH_CLOSE, kernel)
ELSE IF operation == 'Hit':
    binary = convert image to binary
    result = erode the foreground
ELSE IF operation == 'Miss':
    binary = convert image to inverse binary
    result = erode the background
ELSE:
    result = original image

# Save and Display Result
save result as 'output.png' in uploads folder
render original and result image in web interface
```

2.4 Prediction Algorithm

Purpose

- This part handles the application of morphological operations to enhance or modify image structures based on user input. It processes the image by identifying its structural features and applying the selected transformation to highlight or suppress certain elements.

Steps

- **Step 1:** Read the uploaded image using OpenCV and convert it to a suitable format (grayscale or binary, depending on the operation).
- **Step 2:** Retrieve the selected morphological operation (Erosion, Dilation, Opening, Closing, Hit, Miss) and kernel size from the user input form.
- **Step 3:** Create a structuring element (kernel) of the specified size using NumPy to define the shape and extent of the morphological transformation.
- **Step 4:** Apply the corresponding OpenCV function to perform the selected operation:
  - Erosion: Shrinks white areas, removing noise and small objects.
  - Dilation: Expands white areas, emphasizing larger features.
  - Opening: Removes small white noise while preserving overall shape.
  - Closing: Fills small black holes within white regions.
  - Hit: Detects specific foreground structures in binary images.
  - Miss: Detects background features by inverting the binary image.
- **Step 5:** Save the processed image to a designated folder for retrieval.
- **Step 6:** Display the original and transformed images side-by-side on the web interface for visual comparison.

2.5 Data Collection and Analysis Procedure

2.5.1 Data Collection Procedure

Purpose

- To gather a diverse set of input images required to develop, test, and evaluate the effectiveness of various morphological transformations under different visual conditions and use cases.

Sources of Data

1. User-Uploaded Images via GUI:

- Users select images from their local system using a file dialog within the application
- Images include scanned documents, microscopic images, natural scenes, and artificially generated test patterns.
- Enables dynamic testing across multiple categories and file formats (JPEG, PNG, BMP).

2. Predefined Image Sets:

- Used to demonstrate the effectiveness of morphological operations for tasks like noise removal, edge detection, and shape enhancement.
- OpenCV sample images (e.g., lena.jpg, shapes.png).
- Public datasets (e.g., MNIST for binary digits, Bioimage datasets).
- Custom image sets captured using mobile phone or camera for real-world textures and noise.

Capture Conditions

1. Image Variety:

- Binary and grayscale images.
- Text-heavy images (e.g., OCR testing).
- Images with varying levels of noise, blur, and contrast.

2. Noise Simulation:

- Salt-and-pepper noise artificially added to clean images to test erosion and opening.
- Structural holes and gaps simulated to assess closing and dilation performance.

Tools Used for Collection

1. Python + OpenCV:

- Used to capture screenshots, add synthetic noise, convert images to grayscale/binary, and load user images.
- Also used for interactive application of transformations.

2. Photo Editors / Simulators:
    - GIMP, Paint.NET, or PIL (Python Imaging Library) used to create and prepare custom test cases.
3. Smartphone Cameras:
    - Used to take images of text, real-world surfaces, or documents to simulate practical use cases.

2.5.2 Data Analysis Procedure

Purpose

- To evaluate the effectiveness of morphological operations in enhancing image structure, removing noise, and preparing images for subsequent analysis or processing steps.

Analysis Criteria

1. Binary and grayscale images
    - Visual inspection to confirm whether intended changes occurred (e.g., noise removed, edges sharpened).
    - Comparison between original and transformed images to observe clarity and structure enhancement.
2. Parameter Sensitivity (Kernel Size):
    - Effect of varying kernel size from 1×1 to 20×20 on each operation was studied.
    - Large kernels tested for over-smoothing; small kernels for precision.
    - Ideal kernel size range documented for each operation and input type.
3. Operation Comparison:
    - Side-by-side comparisons of Erosion, Dilation, Opening, and Closing on the same image.
    - Observed which operations were best suited for specific tasks like noise reduction or edge enhancement.

4. Noise Handling:

    - Artificial noise (salt-and-pepper, Gaussian blur) added to test images.
    - Erosion and Opening effectiveness evaluated in removing white noise.
    - Closing effectiveness evaluated in removing black gaps in foreground.

Tools for Analysis

1. OpenCV Visualization:

    - Used to display original vs processed images in pop-up windows for direct visual comparison.
    - cv2.imshow() used after each operation to examine the results.
    - Python Timing Functions (Optional):
        - time.time() used to measure processing time for large images or high iteration counts.

2. Matplotlib :

    - Used to display multiple image transformations in a single window for offline analysis.

3. Manual Visual Review:

    - Performed to determine transformation accuracy, clarity improvement, and structural enhancement.
    - Screenshots and logs documented for report writing and demonstration.

Chapter 3

IMPLEMENTATION

3.1 Frontend code (index.html)

Discription

- HTML Structure
  - Uses standard HTML5 structure with !DOCTYPE html,html,head,and body tags.
- Styling
  - Gradient background.
  - Transparent form box.
  - Rounded corners and spacing for clean layout.
  - Button coloring for Submit (green) and Reset (red).
- Form Elements
  - File Upload Input: Appears only if the user hasn’t uploaded an image yet.
  - Dropdown (Select Operation): Lists six morphological operations.
  - Kernel Size Input: Accepts a number from 2 to 20.
  - Submit Button: Changes label based on whether image is uploaded.
  - Reset Button: Allows user to start over and upload a new image.

- Dynamic Content (Jinja Template)
  - Conditional rendering using
  - Whether image is uploaded.
  - What operation is selected.
  - Whether a processed image is available.
- Result Display
  - Original image on the left.
  - Processed image on the right.

```
BEGIN WebPage
  SET language = "en", charset = "UTF-8", title =
  "Morphological Image Processor"
  APPLY styles: gradient background, centered text,
  styled form/buttons/images

  DISPLAY heading: "Morphological Image Processor"

  BEGIN Form (method = POST, file upload enabled)

    IF no image uploaded THEN
      SHOW file input
    ELSE
      SHOW message: "Image loaded. You can change options."
    ENDIF

    SHOW dropdown for operation (Erosion, Dilation, Opening, Closing,
    Hit, Miss)
    SHOW number input for kernel size (2{20)
```

```
    SHOW submit button:
      - "Apply Transformation" if no image
      - "Reapply" if image uploaded

    IF image uploaded THEN
      SHOW "Reset" button linking to reset route
    ENDIF
  END Form

  IF image uploaded AND result exists THEN
    DISPLAY Original and Processed images side by side
  ENDIF
END WebPage
```

3.2 Backend Code (app.py)

Discription

- The Python code in this project forms the backbone of the morphological image processing web application. It utilizes the Flask framework to create a lightweight, server-side application that handles HTTP requests, file uploads, and page rendering. The image processing logic is implemented using OpenCV, a powerful library for computer vision tasks. When a user uploads an image and selects a morphological operation the code reads the image, converts it if necessary, and applies the selected operation using OpenCV functions. The processed image is then saved and displayed alongside the original for comparison. NumPy is used for creating the kernel needed for the morphological operations. The application is modular, with clear separation between the image processing logic and the web interface.

```
    BEGIN Flask Application
```

```
SET secret_key and upload_folder
CREATE upload_folder if not exists

DEFINE process_image(path, operation, kernel_size):
  READ image
  IF image not found → RETURN None
  CREATE kernel of size (kernel_size x kernel_size)

  SWITCH operation:
    CASE 'Erosion' → Apply erosion
    CASE 'Dilation' → Apply dilation
    CASE 'Opening' → Morphological open
    CASE 'Closing' → Morphological close
    CASE 'Hit'/'Miss':
      Convert to grayscale
      Apply binary or inverted threshold
      Apply erosion
      Convert back to color
    DEFAULT → Use original image

  SAVE and RETURN processed image path
END FUNCTION

DEFINE ROUTE '/':
  GET form inputs (operation, kernel)
  IF new image uploaded → Save and store in session
  GET image path from session
  IF image exists → Process image
  RENDER 'index.html' with data
END ROUTE
```

```
DEFINE ROUTE '/reset':
  CLEAR session image
  REDIRECT to home
END ROUTE
RUN app in debug mode
END Flask Application
```

Chapter 4

RESULT

4.1 Input

Figure 4.1: Input car.png

4.2 Output

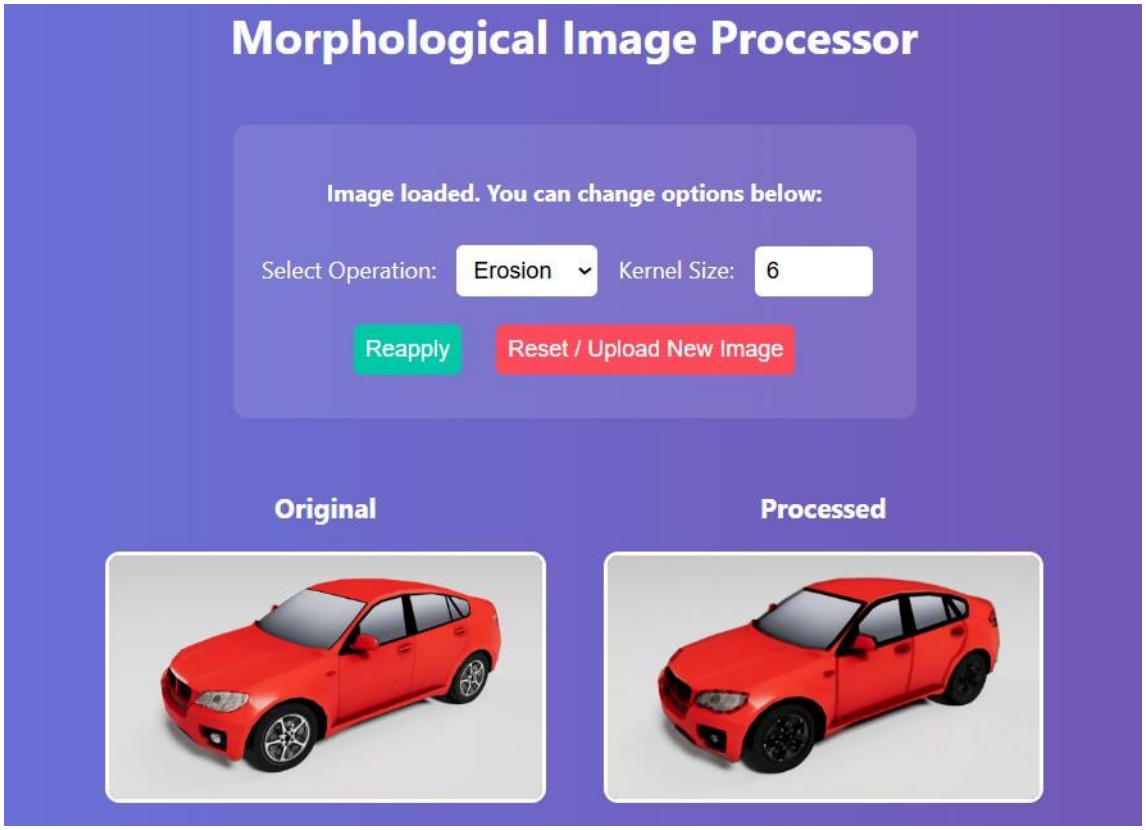

Figure 4.2: Output of Erosion

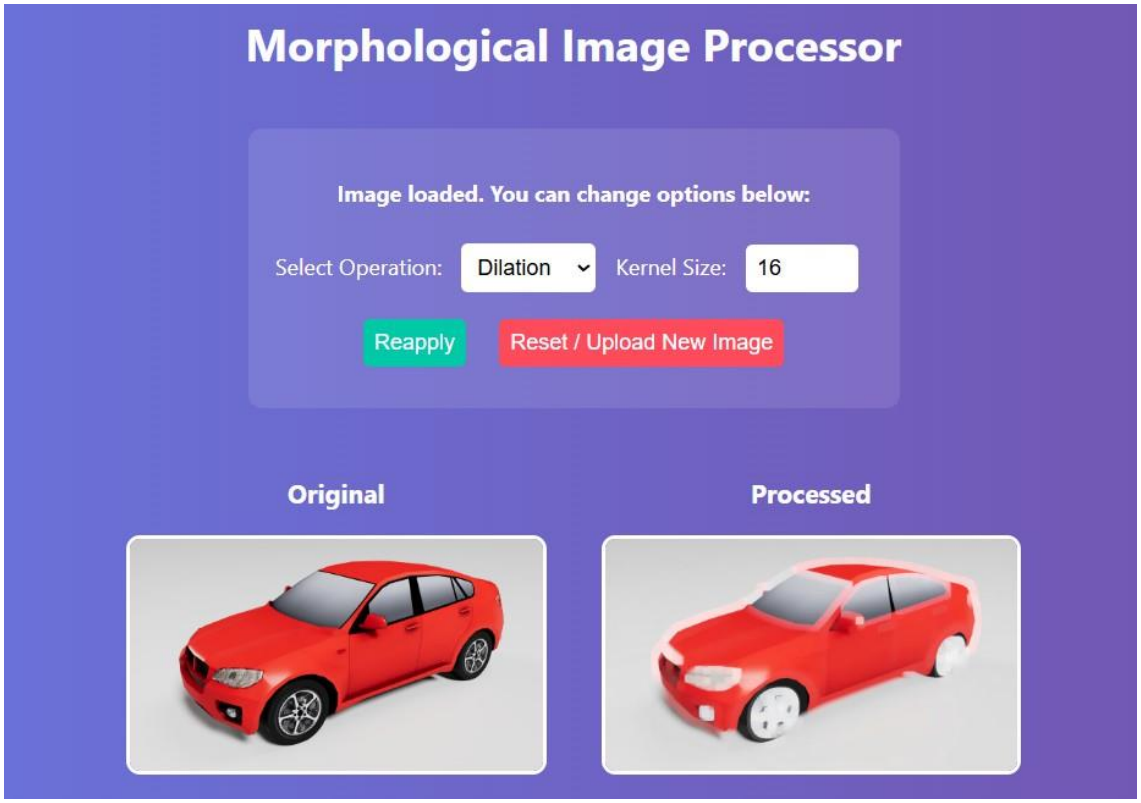

Figure 4.3: Output of Dilation

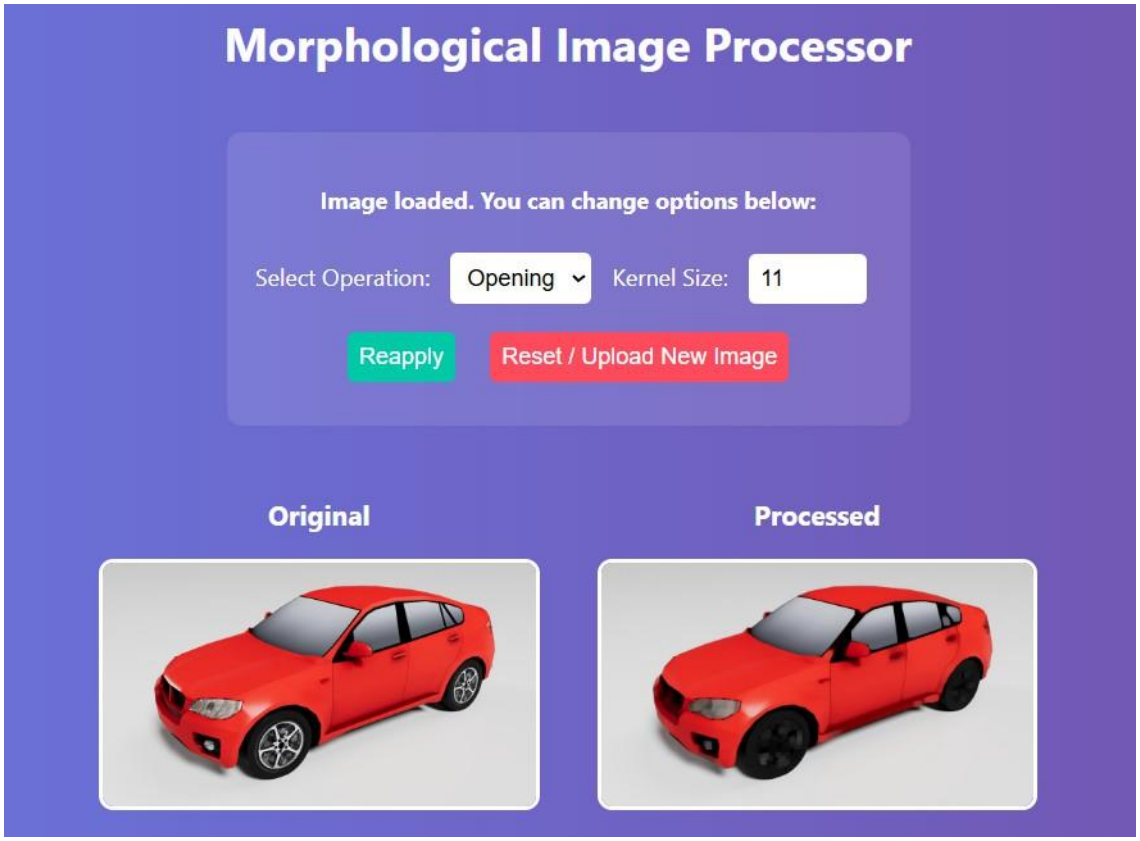

Figure 4.4: Output of Opening

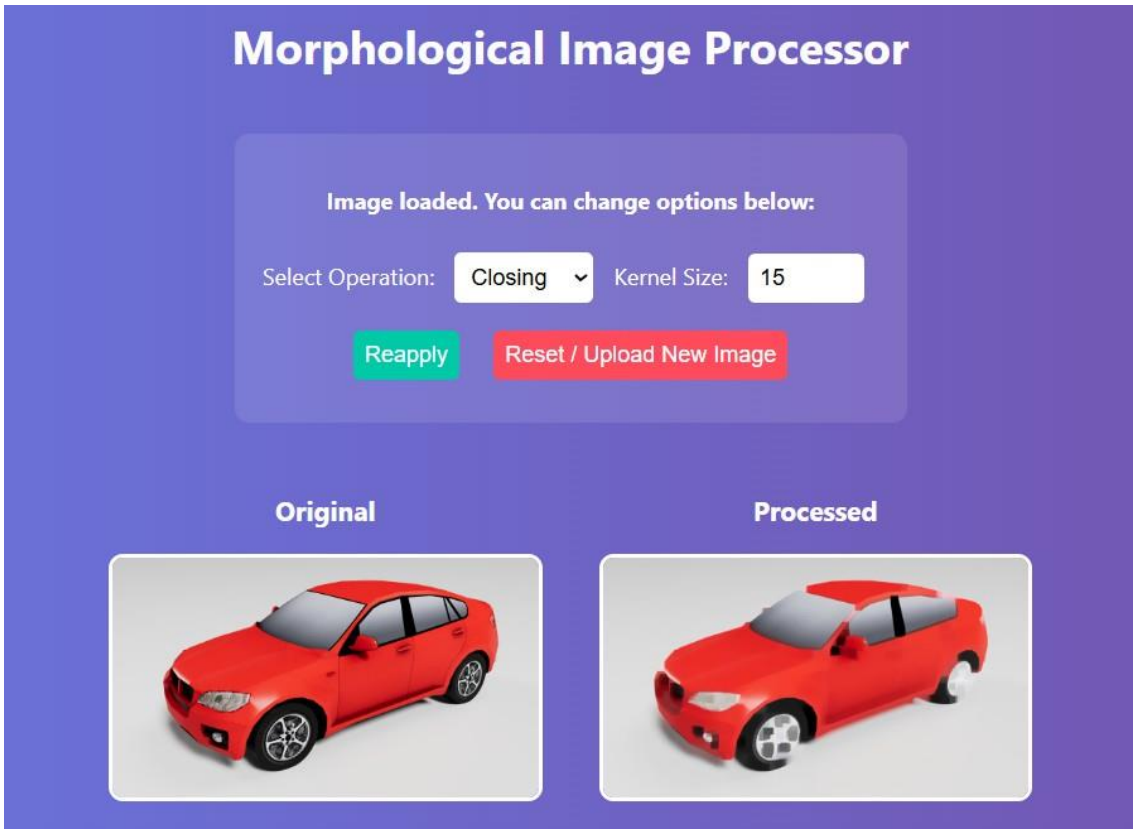

Figure 4.5: Output of Closing

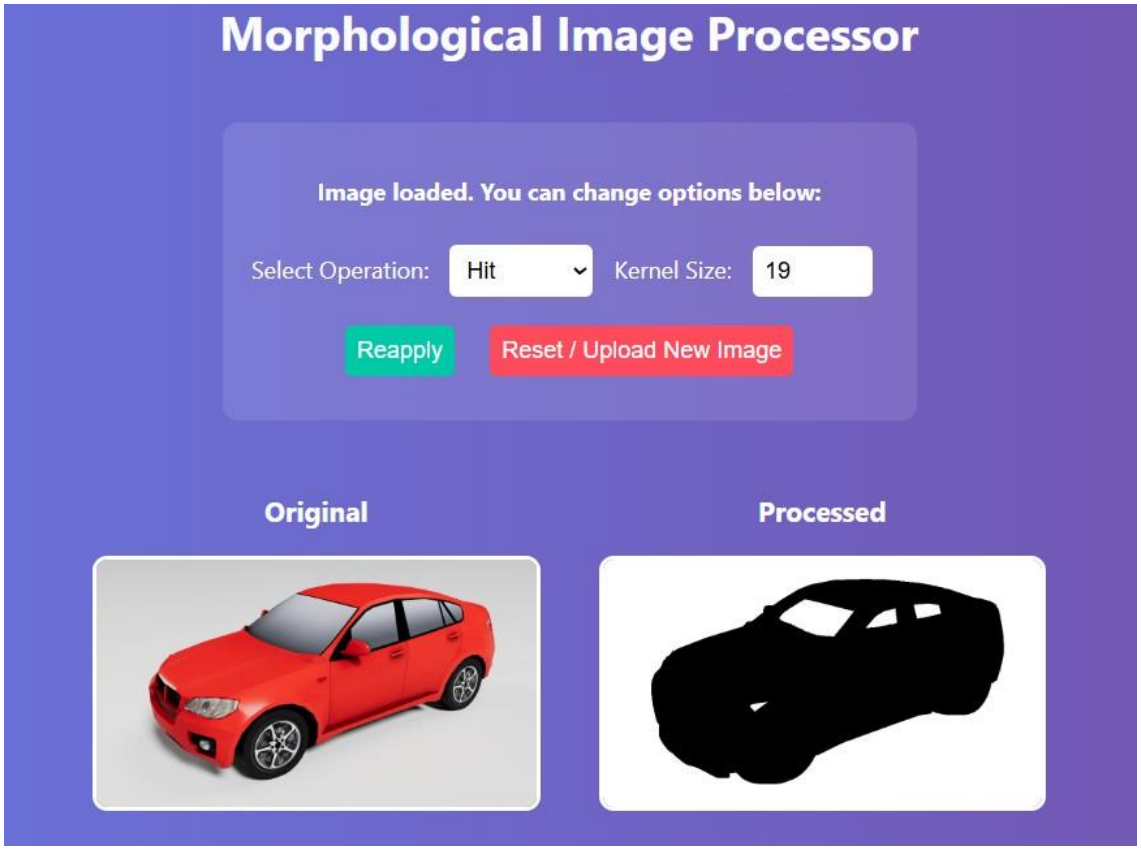

Figure 4.6: Output of Hit

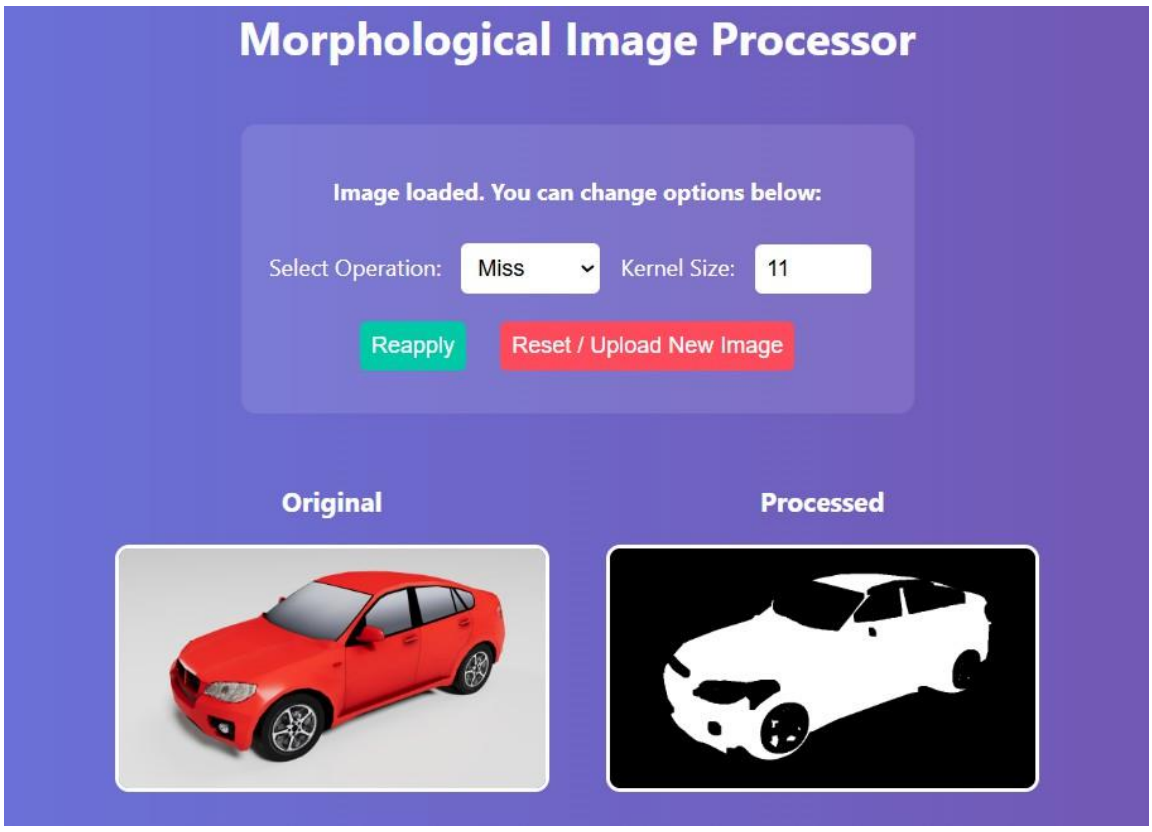

Figure 4.7: Output of Miss

4.3 Outcome

1. Functional Web Application:
   - Successfully developed a web-based platform using Flask and OpenCV for applying morphological image processing operations.
2. User-Friendly Interface:
   - Created an intuitive and interactive interface that allows users to upload images, select operations, and view real-time results without requiring programming skills.
3. Educational Tool:
   - Served as a practical learning aid for students and beginners to understand the effects of morphological transformations like Erosion, Dilation, Opening, Closing, Hit, and Miss.
4. Dynamic Parameter Control:
   - Enabled users to customize kernel sizes, providing flexibility and control over the intensity of each operation.
5. Visual Feedback:
   - Displayed side-by-side comparisons of the original and processed images, enhancing visual learning and understanding of structural image changes.
6. Platform Independence:
   - Ensured compatibility with major operating systems (Windows/Linux) and browsers, allowing broad accessibility.
7. Foundation for Extension:
   - Established a modular structure that can be extended to include advanced operations, real-time video processing, or AI-based recommendations in the future.

Chapter 5

CONCLUSION

5.1 Conclusion

This project successfully demonstrates the implementation of a web-based morphological image processing tool that allows users to apply various structural transformations to digital images. By integrating Flask for the web framework and OpenCV for image manipulation, the system offers a user-friendly platform for performing essential morphological operations such as Erosion, Dilation, Opening, Closing, Hit, and Miss. The application caters to users from academic and non-technical backgrounds, enabling them to experiment with image processing concepts without needing deep programming knowledge or specialized software installations. The real-time processing and side-by-side visualization of the original and transformed images enhance understanding and support interactive learning. Through kernel size customization and operation selection, users gain practical insights into how morphological transformations affect image structure and quality. This platform holds significant potential for educational institutions, research experiments, and early-stage computer vision prototyping. Although currently focused on basic morphological operations, the system lays a solid foundation for future enhancements, such as incorporating advanced processing techniques, real-time video input, and machine learning-driven recommendations. Overall, the project achieves its goal of making shape-based image analysis more accessible, visual, and interactive, while also serving as a stepping stone for more sophisticated computer vision applications.

5.2 Future Enhancements

- Advanced Morphological Operations: Incorporate more complex operations such as skeletonization, edge detection, and watershed segmentation to broaden the scope of analysis.
- Machine Learning Integration: Use AI/ML models to automatically suggest the most suitable morphological operation based on image characteristics.
- Real-Time Video Processing: Extend the system to support real-time morphological transformations on video streams from a webcam or external camera.
- Multi-Channel Image Support: Enable processing for RGB and grayscale images separately, giving users finer control over each channel.
- Download and Export Options: Allow users to download processed images in various formats and resolutions for offline use.
- Mobile and Cross-Platform Compatibility: Optimize the interface for mobile devices and different screen sizes to improve accessibility.
- User Interface Enhancements: Improve the UI/UX with responsive design, progress indicators, and interactive tutorials for beginners.
- Cloud Storage Integration: Provide options to save and retrieve previously processed images using cloud-based storage solutions.
- User Account System: Implement login functionality to personalize the experience and store user history, preferences, and results.
- Educational Mode: Add guided examples and explanations for each operation to support self-learning and classroom use.

References

[1] Gonzalez, R. C., Woods, R. E. (2018). Digital Image Processing (4th ed.). Pearson.

[2] Bradski, G., Kaehler, A. (2008). Learning OpenCV: Computer Vision with the OpenCV Library. O'Reilly Media.

[3] Soille, P. (2003). Morphological Image Analysis: Principles and Applications (2nd ed.). Springer.

[4] Szeliski, R. (2010). Computer Vision: Algorithms and Applications. Springer.

[5] Jähne, B. (2005). Digital Image Processing (6th ed.). Springer.

[6] Shapiro, L. G., Stockman, G. C. (2001). Computer Vision. Prentice Hall.

[7] Otsu, N. (1979). "A Threshold Selection Method from Gray-Level Histograms." IEEE Transactions on Systems, Man, and Cybernetics, 9(1), 62–66.

[8] Zhang, H., Li, P. (2020). "Image Denoising Using Morphological Filter and CNN." Pattern Recognition Letters, 135, 27–34.

[9] OpenCV Documentation. (2023). Morphological Transformations. Retrieved from https://docs.opencv.org/

[10] Kumar, M., Singh, S. K. (2017). "Web-based Image Processing Tools for Educational Applications." International Journal of Computer Applications, 163(6), 12–18.